



SAPIENZA
UNIVERSITÀ DI ROMA

Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci LLM-as-a-Judge per la valutazione del web parsing

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Ingegneria Informatica e Automatica

Ludovica Fiocchetta

Matricola 2103669

Relatore

Professore Roberto Navigli

Anno Accademico 2025/2026

**Oltre il Gold Standard: analisi comparativa tra metriche tradizionali e approcci
LLM-as-a-Judge per la valutazione del web parsing**

Laurea Triennale. Sapienza Università di Roma

© 2026 Ludovica Fiocchetta. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: fiocchetta.2103669@studenti.uniroma1.it

Indice

1	Introduzione	1
1.1	Contesto generale	1
1.2	Motivazione: il progetto Chat Minerva	2
1.3	Obiettivi e contributo dell'elaborato	2
1.4	Struttura della tesi	3
2	Lavori correlati	5
2.1	Tecniche di estrazione	5
2.2	Metriche tradizionali	5
2.3	Metriche semantiche	5
2.4	LLM-as-a-Judge	5
3	Sistema presentato	7
4	Architettura e Implementazione	9
4.1	Panoramica dell'infrastruttura	9
4.2	Progettazione del Backend	9
4.3	Progettazione del Frontend	10
4.4	Gestione dei Dati e Persistenza	10

Capitolo 1

Introduzione

1.1 Contesto generale

Nell'attuale contesto informativo tecnologico, il World Wide Web rappresenta il più vasto bacino di conoscenza non strutturata mai esistito. Con lo sviluppo dei Large Language Models, l'abilità di elaborare e comprendere il linguaggio naturale è diventata una necessità, poiché l'efficacia di questi modelli è fortemente dipendente dalla qualità dei dati forniti in input. Gli LLM hanno ancora una capacità limitata di manipolare con precisione la conoscenza, infatti falliscono nell'accedere a fonti precise e tendono ad avere allucinazioni su compiti *knowledge-intensive*. Le sfide più note includono:

- Presentazione di informazioni false quando non si ha la risposta;
- Presentazione di informazioni obsolete o generiche quando l'utente si aspetta una risposta specifica e attuale;
- Creazione di una risposta da fonti non autorevoli;
- Creazione di risposte imprecise a causa della confusione terminologica, in cui diverse fonti di addestramento utilizzano la stessa terminologia per parlare di argomenti diversi.

Per questi motivi il processo di estrazione di dati puliti assume un ruolo fondamentale. Nel paradigma della *Retrieval-Augmented Generation* (RAG) [1], architettura che ottimizza le prestazioni di un modello di intelligenza artificiale collegandolo a knowledge base esterne, il *web scraping* tradizionale, basato su regole deterministiche e selettori del Document Object Model (DOM), risulta spesso fragile di fronte alla grande diversità delle strutture HTML. Questo approccio spesso lascia insieme al testo utile anche "rumore", come menu di navigazione, banner pubblicitari e riferimenti ad altre pagine, difetto che può tradursi in allucinazioni nelle risposte degli LLM. Un approccio più robusto richiede sistemi di *web parsing* semantico, che consentono di migliorare i risultati essendo in grado di scansionare grandi database e di isolare esclusivamente le informazioni rilevanti nella pagina, ignorando tutti gli elementi di disturbo.

1.2 Motivazione: il progetto Chat Minerva

Questa necessità operativa si manifesta concretamente all'interno del progetto Chat Minerva sviluppato da Sapienza Università di Roma, per il quale è indispensabile fornire al modello testi in input totalmente puliti al fine di garantire risposte attendibili. Chat Minerva è un modello linguistico avanzato che si pone l'obiettivo di fornire un agente di chat affidabile, capace di rispondere in modo accurato e contestualizzato alle molteplici domande da parte degli utenti. Si basa su un'architettura *Retrieval-Augmented Generation* (RAG), così da fare riferimento a una base di conoscenza autorevole al di fuori dai suoi dati di addestramento originali prima di generare una risposta [2], il cui flusso operativo è illustrato in Figura 1.1.

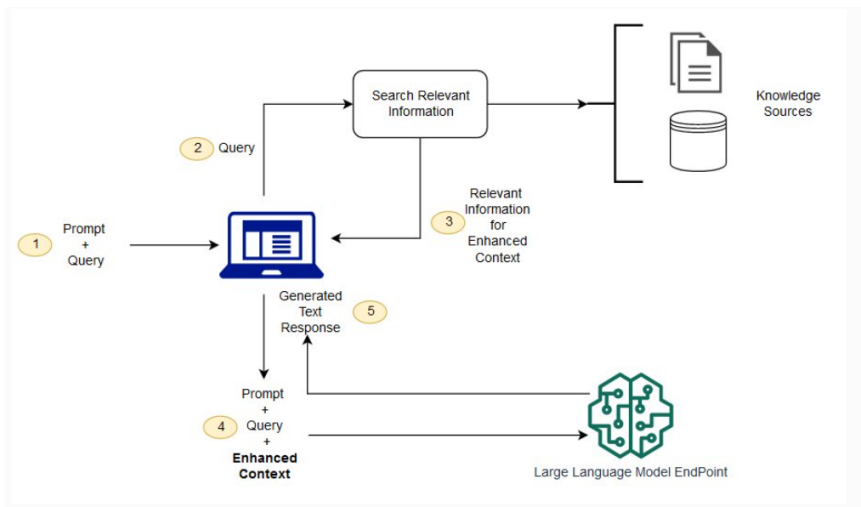


Figura 1.1. Rappresentazione di un'architettura RAG. Il framework sviluppato assicura la qualità dei dati estratti prima del loro inserimento nelle *Knowledge Sources* [2].

Come evidenziato dallo schema, quando l'utente formula una query (passaggi 1 e 2), il sistema ricerca informazioni pertinenti all'interno delle proprie fonti di conoscenza esterne per arricchire il contesto del prompt (passaggi 3 e 4). Se i parser che alimentano questi database estraggono testo sporco o frammenti di codice HTML, l'agente genererà risposte errate o allucinate (passaggio 5). Affinché il modello non generi informazioni sbagliate o non pertinenti è essenziale che i dati forniti in fase di inferenza siano estremamente puliti. Se i parser utilizzati per alimentare Minerva estraggono testo sporco o formattazioni residue, la qualità della risposta del modello degrada drasticamente. È nata quindi la necessità ingegneristica di sviluppare un'infrastruttura capace di testare, misurare e validare in modo oggettivo le performance di diversi sistemi di web parsing, garantendo che al Chatbot venga passata esclusivamente la conoscenza utile e strutturata di cui ha bisogno.

1.3 Obiettivi e contributo dell'elaborato

Il seguente elaborato ha dunque come scopo principale la descrizione di un sistema di estrazione automatizzata di testo pulito da quattro domini internet, e la succes-

siva analisi comparativa sulla valutazione della qualità dei parser. Questo studio metterà a confronto le metriche matematiche tradizionali, che basano il giudizio sulla comparazione con un *gold standard*, con l'innovativo paradigma *LLM-as-a-Judge*, esplorato anche in scenari *reference-free*. Nello specifico, il sistema si articola nei seguenti punti:

- **Sviluppo della pipeline di estrazione e valutazione:** implementazione di un'infrastruttura Python capace di estrarre testo pulito e misurare in modo oggettivo le performance dei sistemi di *web parsing*.
- **Analisi con metriche classiche e avanzate:** valutazione dei *parser* basata sul confronto dei testi con un *gold standard*, calcolando metriche rigorose quali Precision e Recall, ma anche semantiche come BERTScore e METEOR.
- **Sperimentazione del paradigma LLM-as-a-Judge:** introduzione di un approccio moderno in cui un Large Language Model agisce da giudice, esplorando l'efficacia di valutazioni *reference-free* per superare la rigidità delle metriche tradizionali.

Il contributo principale dell'elaborato risiede proprio in questo doppio livello della pipeline di valutazione. Infatti, da un lato si ha un giudizio basato su un *gold standard*, in cui il contenuto estratto viene confrontato con un testo di riferimento salvato all'interno di un database relazionale. In questa fase vengono calcolate metriche classiche basate sulla sovrapposizione insiemistica e sulle distanze operative (come la *Jaccard Similarity*), affiancate da metriche semantiche avanzate (come BERTScore e METEOR). Dall'altro lato, si esplora l'approccio *LLM-as-a-Judge*, chiamando un modello linguistico per valutare la coerenza e la pulizia del testo estratto anche in modalità *reference-free*. Questo permette di superare il limite intrinseco del *gold standard*, che richiede una costosa annotazione manuale preventiva. L'LLM agisce come giudice autonomo: attraverso tecniche di *prompt engineering* e pipeline difensive contro i colli di bottiglia computazionali, è in grado di analizzare la pulizia dell'estrazione anche su domini web sconosciuti, fornendo un punteggio quantitativo e un commento qualitativo pronto all'uso. Questo studio dimostrerà come l'integrazione di approcci semantici avanzati sia essenziale per certificare la qualità del testo estratto, garantendo l'affidabilità di sistemi complessi come Chat Minerva.

1.4 Struttura della tesi

La trattazione è organizzata come segue:

- **Capitolo 2:** analizza i lavori correlati, descrivendo l'evoluzione delle tecniche di *web parsing* e presentando il fondamento teorico delle metriche di valutazione impiegate nel progetto, dagli approcci tradizionali e semantici fino all'introduzione del paradigma *LLM-as-a-Judge*.
- **Capitolo 3:** descrive in dettaglio la progettazione architetturale del sistema, illustrando dettagliatamente tutte le scelte informatiche adottate.

- **Capitolo 4:** esplora l'implementazione degli approcci *LLM-as-a-Judge*, analizzando la costruzione dei *prompt* di valutazione e l'integrazione con i modelli in esecuzione locale.
- **Capitolo 5:** presenta l'impostazione degli esperimenti sui domini selezionati e discute criticamente i risultati ottenuti, confrontando i punteggi delle metriche deterministiche con il giudizio qualitativo dell'intelligenza artificiale.
- **Capitolo 6:** riassume i traguardi raggiunti e delinea i possibili sviluppi futuri per il potenziamento dell'ecosistema.

Capitolo 2

Lavori correlati

2.1 Tecniche di estrazione

2.2 Metriche tradizionali

2.3 Metriche semantiche

2.4 LLM-as-a-Judge

Capitolo 3

Sistema presentato

Capitolo 4

Architettura e Implementazione

4.1 Panoramica dell'infrastruttura

Questo capitolo descrive nel dettaglio l'architettura ingegneristica alla base del framework di valutazione. Il sistema è stato progettato per estrarre in modo automatizzato testo pulito dalle pagine di quattro domini internet di riferimento (*en.wikipedia.org*, *viaggi-usa.it*, *cbsnews.com* e *cnn.com*) e valutarne l'accuratezza tramite il confronto con un *gold standard* generato manualmente.

Per garantire la massima scalabilità, portabilità e riproducibilità dell'ambiente di esecuzione, il progetto adotta un'architettura a microservizi. La logica di estrazione, valutazione e salvataggio dei dati (*backend*) è nettamente separata dall'interfaccia utente (*frontend*). L'intero ecosistema è containerizzato mediante Docker, orchestrato attraverso le specifiche definite nel file `docker-compose.yaml`.

4.2 Progettazione del Backend

Il *backend*, esposto sulla porta 8003, è stato sviluppato seguendo un approccio modulare in Python e si articola nei seguenti macro-componenti fondamentali:

- **Validazione Dati e Server (`server.py`):** sviluppato con il framework FastAPI, questo modulo espone l'interfaccia REST API verso il *frontend*. Sfrutta i modelli Pydantic per una rigorosa validazione dei dati in ingresso e in uscita, assicurando che endpoint cruciali come `/parse`, `/domains` e `/gold_standard` rispettino le specifiche. Un sistema di *routing* avanzato smista dinamicamente le richieste ai diversi *parser* in base al dominio dell'URL fornito.
- **Motore di Estrazione:** implementa una *pipeline* a tre stadi basata sulla libreria Crawl4AI:
 1. *Configurazione e Acquisizione:* impiega `BrowserConfig` in modalità *headless* e `CrawlerRunConfig` con `CacheMode.BYPASS` per garantire l'acquisizione in tempo reale. In alternativa, permette l'estrazione statica diretta fornendo in input una stringa HTML.
 2. *DOM Cleaning tramite JavaScript:* prima della conversione in Markdown, viene iniettato uno script JS mirato per manipolare il DOM e

rimuovere selettivamente elementi di disturbo (infobox, sidebar, footer e banner).

3. *Post-processing Regex*: il testo convertito in Markdown subisce un ulteriore filtraggio tramite espressioni regolari per eliminare residui di tag, metadati e spazi ridondanti.
- **Motore di Valutazione (`accuracy.py`)**: servizio indipendente che riceve le stringhe di testo (estratto e di riferimento) e calcola il grado di sovrapposizione tra i set di *token*. Opera una normalizzazione preventiva (conversione in *lowercase* e rimozione della punteggiatura) per garantire l'assoluta oggettività del confronto.

Completano l'architettura *backend* gli *script* ausiliari `init_db.py` per l'inizializzazione della base di dati e `remove_markdown.py` per la pulizia finale del formato.

4.3 Progettazione del Frontend

L'interfaccia utente, esposta sulla porta 8004, è stata progettata per fornire un punto di accesso intuitivo all'ecosistema di valutazione, articolandosi in due moduli principali:

- **Gestione delle Richieste (`frontend.py`)**: sviluppato anch'esso in FastAPI, agisce da intermediario sfruttando il client asincrono `httpx` per comunicare con il *backend*. Recupera la lista dei domini, interroga i *gold standard* e monitora in tempo reale lo stato dei container (Backend, MariaDB e Ollama). Il modulo coordina tre flussi principali: l'analisi di URL singoli, un pannello CRUD per la gestione interattiva dei dati di riferimento e una *dashboard* per il monitoraggio delle valutazioni.
- **Motore di Rendering (`templates/` e `static/`)**: la presentazione dei dati è affidata al motore Jinja2 per la generazione dinamica dei fogli HTML, stilizzati tramite CSS dedicato. L'interfaccia include una vista comparativa a tre colonne per facilitare il raffronto visivo tra HTML grezzo, *gold standard* e pagina *parsata*, esponendo contestualmente le metriche calcolate.

4.4 Gestione dei Dati e Persistenza

Per garantire la persistenza dei risultati e ottimizzare le risorse di rete, la *pipeline* integra un database relazionale MariaDB. Questa strategia di *caching* locale permette di memorizzare l'HTML delle pagine scansionate, evitando richieste HTTP ridondanti in caso di valutazioni successive sullo stesso URL.

Il database viene inizializzato al primo avvio popolando automaticamente le tabelle di sistema leggendo i file JSON contenuti nella directory `gs_data/`. Lo schema relazionale si articola nelle seguenti tabelle (le chiavi primarie sono evidenziate in grassetto):

- `web_resources` (`url`, `domain`, `title`, `html_text`, `created_at`): memorizza l'HTML grezzo prelevato durante la scansione, fungendo da anagrafica centrale.
- `gold_standard` (`url`, `gold_text`, `created_at`): archivia il testo pulito di riferimento necessario per il calcolo delle metriche di accuratezza. L'attributo `url` funge da chiave esterna.
- `parsed_pages` (`url`, `parsed_text`, `created_at`): conserva il testo elaborato e restituito dal *crawler*.
- `evaluation_results` (`url`, `token_level_metrics`, `llm_judgments`, `evaluated_at`): archivia i risultati finali. Al momento dell'inizializzazione i campi assumono valore `NULL` e vengono popolati dinamicamente. Le metriche insiemistiche vengono salvate in `token_level_metrics`, mentre il voto numerico e il *feedback* testuale dell'intelligenza artificiale risiedono nel campo `llm_judgments`.

Bibliografia

- [1] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.
- [2] Amazon Web Services (AWS). *Cos'è la Retrieval-Augmented Generation (RAG)?*
URL: <https://aws.amazon.com/it/what-is/retrieval-augmented-generation/> (visitato il giorno 31/08/2026).